

cMCP — Engineering Report

A from-scratch Model Context Protocol implementation in pure C11,
measured for spec conformance, performance, and hardening

Generated 2026-06-09 • pure C11, three static link targets, no third-party JSON / MCP SDK

1 Executive summary

cMCP is a from-scratch implementation of the Model Context Protocol (MCP) in pure C11. It is a *library*, not an application: it ships three static link targets (`libcmcp_core.a`, `libcmcp_server.a`, `libcmcp_client.a`) plus reference binaries. The JSON parser, JSON-RPC framing, JSON-Schema subset validator, and all three transports are hand-rolled; the only third-party dependency is `libcurl`, linked by the HTTP *client* alone. This report states what the implementation actually does on the wire, how fast it is against the official reference SDKs, and how it behaves under an adversarial peer — and is careful about *what* each claim licenses.

- **It is spec-faithful, and that is measured, not asserted.** cMCP is cross-checked against the TypeScript reference SDK in *both* wire roles — cMCP-client driving a TS server (32 assertions) and a TS client driving a cMCP server (8 checks) — plus a captured-wire replay gate and a weekly spec-drift watch against the upstream spec.
- **It is fast where it matters and tiny at rest.** On the one apples-to-apples workload (`tools/call echo` over stdio, same client, server swapped) cMCP sustains **43 494 calls/s** — **5.4×** the TypeScript SDK and **45×** the Python SDK — at a p50 of **21 μs**. A stdio server idles at **2.2 MB** RSS, 27–33× lighter than the interpreted SDKs, from a 328 KB static binary.
- **It is hardened against a hostile peer.** A STRIDE threat model drives concrete controls: an SSRF egress guard (DNS-rebinding-safe), schema validation *before* dispatch, parse depth/element caps, stdio and HTTP frame caps, slowloris timeouts plus an accept-rate token bucket, and a credential log-redactor. Each maps to a named test.
- **The full quality gate passes clean.** 3 885 assertions across 30 test binaries, green under Valgrind, ASan/UBSan, and ThreadSanitizer; four libFuzzer harnesses; and a soak driver that shows no drift over long stdio and HTTP runs. Every gate is wired into CI.
- **It is scoped for stdio-primary, single-tenant embedding.** The design point is a host process (*butlerbot*) linking the client library to drive tools over stdio. HTTP is supported and hardened, but the network surface is *defensive depth*; TLS and application-layer authentication are deliberately the reverse proxy’s job (§6), not a “later” item.

2 Conformance: spec-faithful on the wire

For a protocol library the central quality question is not “does it run” but “does it produce *exactly* the bytes the spec requires, and interoperate with an independent implementation.” cMCP answers this with cross-implementation checks in both directions rather than self-consistency alone.

- **cMCP client vs. TS reference server** (`make conformance`, 32 assertions): the cMCP async client runs the full handshake against the official TypeScript SDK server and exercises `tools/list`, `tools/call`, `resources`, and `prompts`.
- **TS client vs. cMCP server** (8 checks): the reference TS client drives the cMCP `echo-server`, confirming the cMCP *server* side is wire-compatible from the other direction.
- **Replay regression gate** (`make replay`): real wire transcripts captured through the `cmcp-tee` proxy are replayed and diffed, so a framing change cannot regress silently.
- **Spec-drift watch** (`make check-spec-drift`, weekly CI): compares the pinned `CMCP_PROTOCOL_VERSION` against the upstream spec directories and flags a divergence before it becomes a bug.

Two conformance details are worth calling out because they are easy to get wrong and are pinned by test:

Dual error channels. A `tools/call` whose arguments fail schema validation surfaces as a *tool-level* result (`{isError:true, content:[...]}`), not a JSON-RPC error — matching the TS reference SDK and wire-compatible with it. The `-32602` error *channel* instead carries protocol-level rejections (unknown tool,

malformed `protocolVersion`). A correct host must check both channels; cMCP pins this with `test_schema` and the host-probe.

Version-tolerant handshake. cMCP pins one protocol version per release but does not abort when a peer advertises a different one: per spec the server responds with success and advertises its own version, and the client captures the server’s version and lets the host decide whether to disconnect. Only a missing or malformed `protocolVersion` yields a hard error.

Schema is a documented subset. The validator covers `type` (incl. array-of-types), `properties`, `required`, `additionalProperties>false`, `enum`, `minLength/maxLength` (Unicode code points), `minimum/maximum`, and `items`. It does *not* implement `$ref`, `oneOf/anyOf/allOf`, `format`, or `pattern` in v0.1 — a deliberate, documented scope (docs/schema-conformance.md), not a silent gap.

3 Performance vs. the state of the art

The comparison below is deliberately apples-to-apples: the *same* cMCP client issues the *same* tools/call echo workload over stdio, and only the subprocess server is swapped per row. All three implementations pay exactly one subprocess hop, so the measurement isolates the server runtime. Numbers are 10 000 iterations after a 1 000-iteration warmup on the development machine (`make bench-compare`, 2026-06-08).

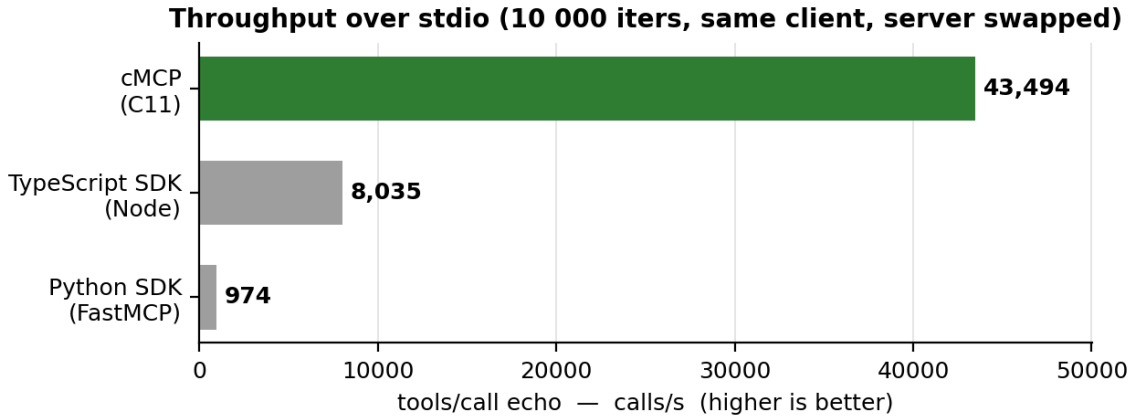


Figure 1. Throughput on the one apples-to-apples engine comparison. Green is this work; grey are the official reference SDKs running the identical client and workload.

Table 1. Throughput, latency, and idle footprint. Latency is per round-trip; RSS is an idle stdio server.

Server	calls/s	vs cMCP	p50 (µs)	p99 (µs)	idle RSS
cMCP (C11)	43 494	1.0×	21	31	2.2 MB
TypeScript SDK (Node)	8 035	0.18×	95	189	74 MB
Python SDK (FastMCP)	974	0.02×	994	1 315	60 MB

3.1 Why the gap — and what the comparison licenses

The spread is a *runtime* story, not a protocol one. cMCP is native C11 with no interpreter, no GC, and (for a stdio server) zero third-party dependencies: the server artifact is a 328 KB static binary. The TypeScript SDK carries a Node runtime plus `node_modules` (`sdk`, `zod`); the Python SDK carries CPython plus a `venv`. The honest reading is that this is an **engine / footprint comparison**, exactly the regime where a compiled language wins, and *not* a claim of feature parity — the reference SDKs ship a richer ecosystem (decorators, type-driven schema, framework integrations) that cMCP intentionally does not. What the chart does license is the deployment claim: at the *butlerbot* embedding point, the protocol layer costs microseconds and a couple of megabytes, so it disappears inside an LLM inference cycle.

For reference, cMCP’s pure in-process ceiling (no subprocess, inline pipe pair) is **50 487 calls/s at p50 = 19 µs**; the cross-SDK rows each pay the subprocess hop so all three are measured identically. Because hardware and kernel move these numbers, a weekly `bench-compare` CI job re-runs the comparison and posts a fresh table to its run summary, so the published snapshot never silently goes stale.

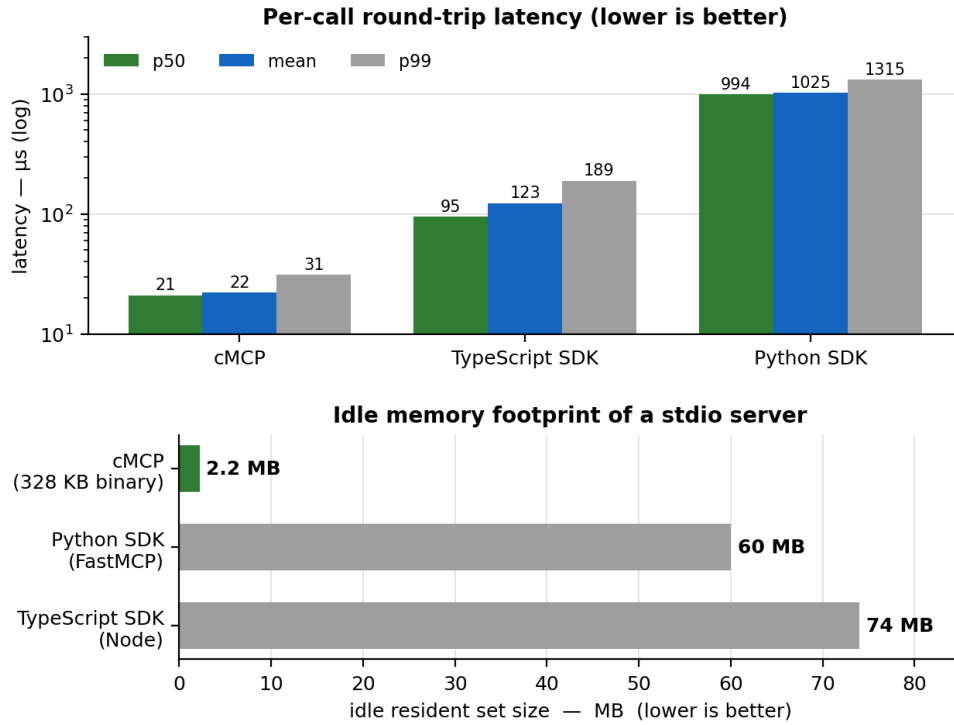


Figure 2. Per-call latency (log axis) and idle memory footprint. cMCP’s p99 (31 μs) sits below the other implementations’ p50; its idle RSS is a rounding error next to an interpreter and its `node_modules/venv`.

4 Test posture and quality gates

cMCP’s suites map onto the standard Unit → Integration → Evaluation pyramid, with a security/hardening axis that cuts across all layers. The hermetic `make test` run is 3 885 assertions across 30 binaries.

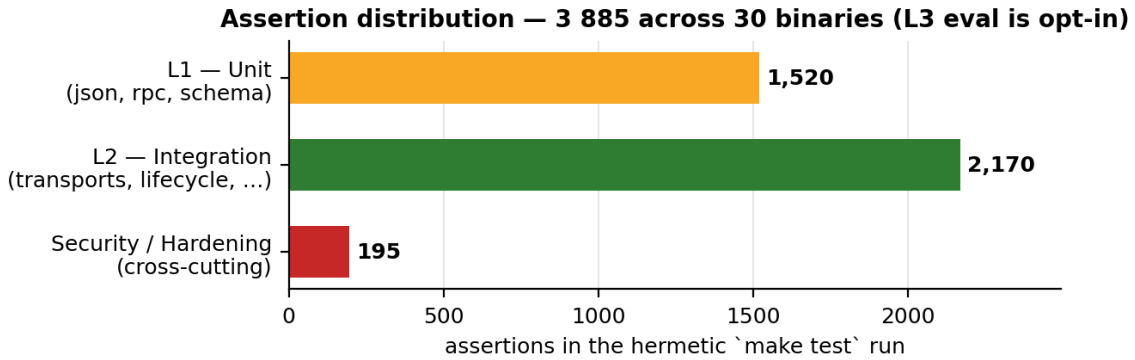


Figure 3. Where the assertions live. L3 (cross-implementation evaluation) is opt-in — it needs Node and network — so it is excluded from the hermetic count and reported separately in §2.

The integration tier is not mocked: `test_stdio_roundtrip` actually `fork()`s a child server and runs a real handshake, and `test_client_server` wires real `cmcp_server_run` instances onto pipe pairs to exercise the async client and multi-server session end-to-end. The concurrency model under test is one reader thread per transport, handlers on a small worker pool, and a single writer mutex per transport — TSan-clean.

5 Robustness and security (hardening)

cMCP is a protocol library, so the trust model is precise: the host is trusted (cMCP runs in its address space), but the *peer* on the other end of a transport may be hostile or buggy and its input is untrusted. The hardening maps a STRIDE threat catalogue onto concrete, individually-tested controls.

The controls are layered front-to-back: a request crosses the `Origin` allow-list and SSRF guard, then frame /

Table 2. Quality gates, all wired into CI; green as of 2026-06-08.

Gate	Command	What it proves
Unit + integration	<code>make test</code>	3 885 assertions / 30 binaries
Memory safety	<code>make valgrind</code>	leak-free, no invalid access
ASan + UBSan	<code>make test-asan</code>	no heap / undefined-behaviour errors
ThreadSanitizer	<code>make test-tsan</code>	no data races
Fuzzing	<code>make fuzz-smoke</code>	4 libFuzzer harnesses (json/rpc/schema/http)
Wire regression	<code>make replay</code>	captured-transcript replay diff
Soak (stdio + HTTP)	<code>make soak</code>	no drift over long runs
Cross-impl conformance	<code>make conformance</code>	matches TS reference SDK, both roles
Spec-drift watch	<code>make check-spec-drift</code>	pinned protocol vs. upstream spec

Table 3. Threat → control → test. Every control is reachable from an environment knob and pinned by a named suite.

Threat	Control	Test
SSRF via discovery	egress guard on the <i>resolved</i> peer (rejects meta-data / link-local / RFC1918 / CGNAT / ULA); DNS-rebinding-safe	<code>test_http_ssrf_guard</code>
DNS rebinding	Origin allow-list on the HTTP server	<code>test_http_server</code>
Insecure deserialization	schema validation <i>before</i> dispatch; parse depth + per-container element caps	<code>test_schema</code> , <code>test_hostile_peer</code>
Memory-bomb / OOM	stdio frame cap, HTTP body cap, handler RLIMIT_AS	<code>test_stdio_frame_cap</code> , <code>test_handler_rlimit</code>
Slowloris / spike	accept-rate token bucket, per-recv idle timeout + cumulative deadline	<code>test_hardening_cross_knob</code>
Credential leakage	log redactor over credential-shaped keys	<code>test_http_auth</code> , <code>test_logging</code>
Malformed / hostile peer	bounded parsers, fail-closed framing	<code>test_hostile_peer</code>

body / depth / element caps, then schema validation, then rate-limit and idle/deadline timeouts, before a handler runs on the worker pool under RLIMIT_AS; anything the handler logs passes the credential redactor on the way out. Each is governed by a snapshot-once environment knob (e.g. CMCP_HTTP_ALLOW_PRIVATE, CMCP_JSON_MAX_DEPTH, CMCP_STDIO_MAX_FRAME, CMCP_HTTP_ACCEPT_RATE, CMCP_LOG_REDACT), so an operator can tune or disable a control for an environment they own without recompiling. Beyond the unit gates, four libFuzzer harnesses drive the JSON, JSON-RPC, schema, and HTTP-head parsers against curated corpora, and the soak driver runs the stdio and HTTP servers for long windows (with optional server respawn) to prove no leak or drift accumulates.

6 Operating envelope — scope and deliberate non-goals

A report owes an honest answer to “what is this *not* for.” cMCP’s design point is a host process linking the client library to drive one or more MCP servers over stdio, single-tenant. The network surface exists and is hardened, but two large areas are deliberately out of scope — by architecture, not by omission.

TLS is terminator-only. cMCP links no TLS library. A protocol library is the wrong layer to own cert rotation, OCSP stapling, and session resumption; the state-of-the-art shape for a JSON-RPC server is behind nginx / caddy / Envoy, which already do TLS, HTTP/2, and rate-limiting better than a single-purpose library could, and linking a TLS stack would force one implementation choice on every downstream. This is an architectural decision for cMCP’s lifetime, not a “TLS later” position; the proxy recipes (including X-Forwarded-Proto trust) live in docs/deployment-tls.md.

Application-layer authentication is the terminator’s job. OAuth2.1 / API keys / mTLS user identification happen at the proxy; cMCP receives the already-authenticated request and trusts it. Likewise out of scope: handler-level resource exhaustion (disk, egress, child processes — the handler decides what is

safe), cryptographic correctness beyond the single CSPRNG read used to mint session ids, and timing side channels (constant-time session-id compare is not implemented, a documented low-risk gap that requires near-position privilege to exploit).

cMCP is intentionally *not* cRAG-shaped: cRAG is one example consumer (wrapped by the in-tree `crag-mcp` reference server), not a privileged dependency, and no cRAG-specific code path lives inside `src/`.

7 Methodology and honest caveats

- **Performance numbers are machine-specific.** They come from `make bench-compare` on the development box; kernel, build flags, and hardware move them. The weekly `bench-compare` CI job re-runs the comparison so the snapshot stays current; treat absolute values as indicative and the *ratios* as the durable signal.
- **The cross-SDK comparison is engine-level.** All three rows run the identical client and workload and pay one subprocess hop, so the comparison is fair — but it measures runtime and footprint, not feature parity. The reference SDKs offer ecosystem conveniences cMCP does not.
- **L3 evaluation is opt-in.** The cross-implementation conformance and replay checks need Node and network, so they sit outside the hermetic `make test` gate and are reported separately. The 3 885-assertion figure is the offline floor, not the ceiling.
- **The schema validator is a documented v0.1 subset.** Unsupported keywords (`$ref`, `oneOf/anyOf/allOf`, `format`, `pattern`) are listed, not silently dropped.
- **The threat model is stdio-primary.** HTTP-axis controls (SSRF, origin, slowloris, accept-rate) are defensive depth for a surface that, in the target deployment, sits behind a trusted terminator.

cMCP — pure C11, three static link targets, no third-party JSON / MCP SDK. All figures reproducible: `make test`, `make bench-compare`, `make conformance`, `make replay`.